

DESIGN AND IMPLEMENTATION OF A TURKISH INSPIRED PROGRAMMING LANGUAGE, LEXICAL AND SYNTAX ANALYZER

Course: Programming Languages / Compiler Design

Term Project Report

Group Members and Student IDs:

Rozerin Yaşar- [230315082]

Cavit Can Key- [230316030]

Azize Görür- [220316018]

Yiğit Kızıldağ- [230316021]

Due Date: 01.06.2026

NAME OF LANGUAGE: "NOWTURK"

1. INTRODUCTION

This project was carried out to demonstrate the practical application of the lexical analysis and syntax analysis phases, which are of critical importance in computer science and programming language theory. Within the scope of the project, an original, lightweight programming language with Turkish keywords and character support was designed; additionally, a functional compiler front-end capable of analyzing the source code for this language was developed. The developed language includes intuitive keywords that allow developers to write code using their native terminology. The main objective is to concretize how a source text is converted into token sequences, how the language's grammatical compliance is checked using the top-down recursive-descent parsing method, and how variable management is maintained through a symbol table.

2. BASE TEXTBOOK CODE USED

The architectural structure and algorithm foundations of the project are based on the sample codes presented in "Chapter 4: Lexical and Syntax Analysis" of Robert W. Sebesta's "Concepts of Programming Languages" (10th Edition). The following adaptations were made in accordance with the project requirements:

What We Kept: The global variable architecture of the lexical analyzer starting on page 172 (**charClass**, **lexeme**, **nextChar**, **nextToken**, **lexLen**) and the basic character classes (**HARF**, **RAKAM**, **BILINMEYEN**) were preserved. The expression (**ifade()**), **term** (**terim()**), and **factor** (**faktor()**) function chain found in the recursive-descent parser template on page 182, along with their synchronous execution with the **lex()** function, were used as the foundation.

What We Modified:

```
/* Token codes */
#define INT_LIT 10
#define IDENT 11
#define ASSIGN_OP 20
#define ADD_OP 21
#define SUB_OP 22
#define MULT_OP 23
#define DIV_OP 24
#define LEFT_PAREN 25
#define RIGHT_PAREN 26
```

```
/* Token Codes */
static final int TAMSAYI_SBT = 10;
static final int KIMLIK = 11;
static final int ATAMA_OP = 20;
static final int TOPLA_OP = 21;
static final int ÇIKAR_OP = 22;
static final int CARP_OP = 23;
static final int BOL_OP = 24;
static final int SOL_PRTZ = 25;
static final int SAG_PRTZ = 26;
static final int SAYI_KW = 30;
static final int EGER_KW = 31;
static final int DONGU_KW = 32;
static final int HARF_KW = 33;
static final int DEĞİLSE_KW = 34;
static final int NOKTALI_V = 35;
static final int ESIT_OP = 40; // ==
static final int DEĞİL_ESIT = 41; // !=
static final int KUCUK_ESIT = 42; // <=
static final int BUYUK_ESIT = 43; // >=
static final int KUCUK_OP = 44; // <
static final int BUYUK_OP = 45; // >
static final int SOL_SUSLU = 50; // {
static final int SAG_SUSLU = 51; // }
static final int YAZDIR = 52;
```

The English keywords and character reader functions in the original code were updated to recognize Turkish character sets (ç, ğ, ı, ö, ş, ü, Ç, Ğ, İ, Ö, Ş, Ü).

```
// Keywords are updated with full Turkish characters
if (kelime.equals("sayı")) nextToken = SAYI_KW;
else if (kelime.equals("eğer")) nextToken = EGER_KW;
else if (kelime.equals("döngü")) nextToken = DONGU_KW;
else if (kelime.equals("harf")) nextToken = HARF_KW;
else if (kelime.equals("değilse")) nextToken = DEGILSE_KW;
else if (kelime.equals("yazdır")) nextToken = YAZDIR;
else nextToken = KIMLIK;
break;
```

The lookup() function was expanded so that, in addition to single character operators, two-character relational operators (==, !=, <=, >=) are accurately recognized using a lookahead method (nextChar = getc(in_fp)).

```
static int lookup(char ch) {
    try {
        switch (ch) {
            case ';': addChar(); nextToken = NOKTALI_V; break;
            case '=':
                addChar();
                int next = in_fp.read();
                if (next == '=') {
                    nextChar = (char) next;
                    addChar();
                    nextToken = ESIT_OP;
                } else {
                    if (next != -1) in_fp.unread(next);
                    nextToken = ATAMA_OP;
                }
                break;
            case '{': addChar(); nextToken = SOL_SUSLU; break;
            case '}': addChar(); nextToken = SAG_SUSLU; break;
            case '!':
                addChar();
                next = in_fp.read();
                if (next == '=') {
                    nextChar = (char) next;
                    addChar();
                    nextToken = DEGIL_ESIT;
                } else {
                    if (next != -1) in_fp.unread(next);
                    nextToken = BILINMEYEN;
                }
                break;
        }
    }
}
```

What We Added:

The **program()** function to manage the program's main body

```
static void program() {
    System.out.println("Giriş <program>");
    if (nextToken == KIMLIK && lexeme.toString().equals("ana")) {
        lex();
        if (nextToken == SOL_PRTZ) {
            lex();
            if (nextToken == SAG_PRTZ) {
                lex();
                if (nextToken == SOL_SUSLU) {
                    lex();
                    komutlar();
                    if (nextToken == SAG_SUSLU) {
                        lex();
                    } else { hata("Program sonunda '}' eksik!"); }
                } else { hata("Program başlangıcında '{' eksik!"); }
            } else { hata("ana() ifadesinde ')' eksik!"); }
        } else { hata("ana() ifadesinde '(' eksik!"); }
    } else { hata("Program 'ana()' fonksiyonu ile başlamalıdır!"); }
    System.out.println("Çıkış <program>");
}
```

komutlar() to process command sequences:

```
static void komutlar() {
    System.out.println("Giriş <komutlar>");
    while (nextToken != SAG_SUSLU && nextToken != EOF_CODE) {
        komut();
    }
    System.out.println("Çıkış <komutlar>");
}

static void komut() {
    System.out.println("Giriş <komut>");
    if (nextToken == SAYI_KW || nextToken == HARF_KW) {
        tanımlama();
    } else if (nextToken == KIMLIK) {
        atama();
    } else if (nextToken == EGER_KW) {
        eger_stmt();
    } else if (nextToken == DONGU_KW) {
        dongu_stmt();
    } else if (nextToken == YAZDIR) {
        yazdir_stmt();
    } else {
        hata("Geçersiz komut yapıları!");
        lex();
    }
    System.out.println("Çıkış <komut>");
}
```

the **eger_stmt()** and **dongu_stmt()** functions to evaluate control flows were added to the system

```
static void dongu_stmt() {
    System.out.println("Giriş <dongu_stmt>");
    if (nextToken == DONGU_KW) {
        lex();
        if (nextToken == SOL_PRTZ) {
            lex();
            koşul();
            if (nextToken == SAG_PRTZ) {
                lex();
                blok();
            } else { hata("Döngü koşulunda ')' eksik!"); }
        } else { hata("Döngü koşulunda '(' eksik!"); }
    }
    System.out.println("Çıkış <dongu_stmt>");
}
```

```
static void eger_stmt() {
    System.out.println("Giriş <eger_stmt>");
    if (nextToken == EGER_KW) {
        lex();
        if (nextToken == SOL_PRTZ) {
            lex();
            koşul();
            if (nextToken == SAG_PRTZ) {
                lex();
                blok();
                if (nextToken == DEGILSE_KW) {
                    lex();
                    blok();
                }
            } else { hata("Eğer koşulunda ')' eksik!"); }
        } else { hata("Eğer koşulunda '(' eksik!"); }
    }
    System.out.println("Çıkış <eger_stmt>");
}
```

```
static void tanimlama() {
    int secilenTip = nextToken;
    if (nextToken == SAVI_KW || nextToken == HARF_KW) {
        lex();
        if (nextToken == KIMLIK) {
            sembolEkle(lexeme.toString(), secilenTip);
            lex();
            if (nextToken == NOKTALI_V) {
                lex();
            } else {
                hata("Tanımlama sonunda ';' eksik!");
            }
        } else {
            hata("Veri tipinden sonra değişken ismi (kimlik) bekleniyor!");
        }
    } else {
        hata("Geçersiz veri tipi! 'sayı' veya 'harf' bekleniyordu.");
    }
    System.out.println("Çıkış <tanimlama>");
}

static void atama() {
    System.out.println("Giriş <atama>");
    if (nextToken == KIMLIK) {
        lex();
        if (nextToken == ATAMA_OP) {
            lex();
            ifade();
            if (nextToken == NOKTALI_V) {
                lex();
            } else { hata("Atama cümlesi sonunda ';' eksik!"); }
        } else { hata("Atama operatörü '=' bekleniyor!"); }
    }
    System.out.println("Çıkış <atama>");
}
```

```
static void yazdir_stmt() {
    System.out.println("Giriş <yazdir_stmt>");
    if (nextToken == YAZDIR) {
        lex();
        if (nextToken == SOL_PRTZ) {
            lex();
            ifade();
            if (nextToken == SAG_PRTZ) {
                lex();
                if (nextToken == NOKTALI_V) {
                    lex();
                } else { hata("Yazdır komutu sonunda ';' eksik!"); }
            } else { hata("Yazdır fonksiyonunda ')' eksik!"); }
        } else { hata("Yazdır fonksiyonunda '(' eksik!"); }
    }
    System.out.println("Çıkış <yazdir_stmt>");
}

static void koşul() {
    System.out.println("Giriş <koşul>");
    ifade();
    if (nextToken == ESIT_OP || nextToken == DEGIL_ESIT ||
        nextToken == KUCUK_OP || nextToken == BUYUK_OP ||
        nextToken == KUCUK_ESIT || nextToken == BUYUK_ESIT) {
        lex();
        ifade();
    } else { hata("Geçersiz karşılaştırma operatörü!"); }
    System.out.println("Çıkış <koşul>");
}
```

```
static void blok() {
    System.out.println("Giriş <blok>");
    if (nextToken == SOL_SUSLU) {
        lex();
        komutlar();
        if (nextToken == SAG_SUSLU) {
            lex();
        } else { hata("Blok sonunda ')' eksik!"); }
    } else { hata("Blok başlangıcında '{' eksik!"); }
    System.out.println("Çıkış <blok>");
}
```

tanımlama() (Declaration): Handles variable declarations by verifying valid data types (sayı, harf), registering the specific identifier into the symbol table, and ensuring the statement correctly ends with a semicolon.

atama() (Assignment): Processes assignment operations by strictly verifying that a valid identifier is followed by an assignment operator (=), an evaluated expression, and a terminating semicolon.

yazdır_stmt() (Print Statement): Parses the custom output command (yazdır), ensuring the expression to be printed is structurally enclosed within parentheses and properly terminated.

koşul() (Condition): Evaluates logical test conditions for loops and if-statements by expecting two distinct valid expressions separated by a relational operator (e.g., ==, <, >=). **blok() (Block):** Defines local execution scopes by enforcing that a sequence of commands (komutlar) is strictly contained within matching left and right curly braces ({ }).

3. Overview of the Designed Language

In this project, we designed a simple, structured programming language that uses Turkish keywords to make it feel more familiar. Our main goal was to connect formal compiler rules with Turkish word structures. The language is strongly-typed and imperative, meaning that variables have strict types and the code runs step-by-step from top to bottom, just like in C or Java.

Our language supports two main primitive data types: **sayı** for storing **integers** and **harf** for storing single characters. To make sure variables don't conflict with each other, we implemented block scoping using standard curly **braces {}**. This ensures that a variable created inside a specific block cannot be accessed from the outside. Finally, every program must start inside a mandatory main function called **ana()**, which holds all the loops, conditions, and variable declarations together.

Quick Dictionary of Our Language

ana()	main()
sayı	int
harf	char
eğer	if
değilse	else
döngü	while
yazdır()	print()
hata	error

Reserved Keywords and Tokens

Token Name	Token Code	Description / Matching Pattern or Keyword
HARF	0	Alphabet letters and Turkish characters (ç, ğ, ...))
RAKAM	1	Numeric characters between 0-9
BILINMEYEN	99	Unknown operators and separators outside defined classes
TAMSAYI_SBT	10	Integer constants (e.g., 13, 28, 45)
KIMLIK	11	User-defined identifiers (Variables and functions)
ATAMA_OP	20	Assignment operator (=)
TOPLA_OP	21	Addition operator (+)
CIKAR_OP	22	Subtraction operator (-)
CARP_OP	23	Multiplication operator (*)
BOL_OP	24	Division operator (/)
SOL_PRTZ	25	Left parenthesis (()
SAG_PRTZ	26	Right parenthesis ())
SAYI_KW	30	Integer variable type declaration keyword (sayı)
EGER_KW	31	Conditional branch keyword (eğer)
DONGU_KW	32	Looping construct keyword (döngü)
HARF_KW	33	harf Keyword (Data Type)
DEGILSE_KW	34	Alternative conditional branch keyword (değilse)
NOKTALI_V	35	Statement delimiter / Semicolon (;)
ESIT_OP	40	Equality comparison operator (==)
DEGIL_ESIT	41	Inequality comparison operator (!=)
KUCUK_ESIT	42	Less than or equal to operator (<=)
BUYUK_ESIT	43	Greater than or equal to operator (>=)
KUCUK_OP	44	Strictly less than operator (<)
BUYUK_OP	45	Strictly greater than operator (>)

SOL_SUSLU	50	Left curly brace / Block opening ({)
SAG_SUSLU	51	Right curly brace / Block closing (})
YAZDIR	52	Standard print command keyword (yazdır)
EOF	-1	End of File indicator

4.BNF and EBNF Grammar Rules

BNF:

- `<program> ::= ana() { <komutlar> }`
- `<komutlar> ::= <komut> | <komut> <komutlar>`
- `<komut> ::= <tanimlama> | <atama> | <eger_stmt> | <dongu_stmt> | <yazdir_stmt>`
- `<tanimlama> ::= <tip> <kimlik>;`
- `<tip> ::= sayı | harf`
- `<atama> ::= <kimlik> = <ifade>;`
- `<eger_stmt> ::= eğer ( <koşul> ) <blok> | eğer ( <koşul> ) <blok> değilse <blok>`
- `<dongu_stmt> ::= döngü ( <koşul> ) <blok>`
- `<yazdir_stmt> ::= yazdır ( <kimlik> );`
- `<koşul> ::= <ifade> <karsilastirma> <ifade>`
- `<karsilastirma> ::= == | != | > | < | >= | <=`
- `<blok> ::= { <komutlar> }`
- `<ifade> ::= <terim> | <terim> + <ifade> | <terim> - <ifade>`
- `<terim> ::= <faktor> | <faktor> * <terim> | <faktor> / <terim>`
- `<faktor> ::= <kimlik> | <tamsayi_sbt> | ( <ifade> )`
- `<kimlik> ::= <harf> { <harf> | <rakam> }`
- `<tamsayi_sbt> ::= <rakam> { <rakam> }`
- `<harf> ::= a | b | c | ... | y | z | A | B | C | ... | Y | Z | ç | ğ | ı | ö | ş | ü | Ç | Ğ | İ | Ö | Ş | Ü`
- `<rakam> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

EBNF

- `program = "ana()" "{" komutlar "}" ;`
- `komutlar = { komut } ;`
- `komut = tanimlama | atama | eger_stmt | dongu_stmt | yazdir_stmt;`
- `tanimlama = tip kimlik ";" ;`
- `tip = "sayı" | "harf" ;`
- `atama = kimlik "=" ifade ";" ;`
- `eger_stmt = "eğer" "(" koşul ")" blok [ "değilse" blok ] ;`
- `dongu_stmt = "döngü" "(" koşul ")" blok ;`
- `yazdir_stmt = "yazdır" "(" kimlik ")" ";" ;`

- koşul = ifade karsilastirma ifade ;
 - karsilastirma = "==" | "!=" | ">" | "<" | ">=" | "<=" ;
 - blok = "{" komutlar "}" ;
 - ifade = terim { ("+" | "-") terim } ;
 - terim = faktor { ("*" | "/") faktor } ;
 - faktor = kimlik | tamsayi_sbt | "(" ifade ")" ;
 - kimlik = harf { harf | rakam } ;
 - tamsayi_sbt = rakam { rakam } ;
 - harf = "a" | "b" | ... | "z" | "A" | "B" | ... | "Z" | "ç" | "ğ" | "ı" | "ö" | "ş" | "ü" | "Ç" | "Ğ" | "İ" | "Ö" | "Ş" | "Ü" ;
 - rakam = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
-

5.LEXICAL ANALYZER DESIGN

The primary function of the lexical analyzer is to continuously read the character stream and segment it into meaningful tokens. The system operates as follows:

Character Reading and Turkish Support (getChar): Characters are retrieved sequentially from the file stream using the `in_fp.read()` method. The analyzer identifies whether the character is a standard alphabet letter via `Character.isLetter()`, while simultaneously checking if it belongs to the specific Turkish character set using `"çğİöŞÜÇĞİÖŞÜ".indexOf()`. If either condition is satisfied, the `charClass` is strictly assigned to **HARF**. For numeric values, which are evaluated via the `Character.isDigit()` function, the class is set to **RAKAM**. Any unmapped or unrecognized symbols are systematically marked as **BILINMEYEN**, and end-of-file stream states are safely handled by explicitly assigning the **EOF_CODE**.

```
/* getChar - Character reader with Turkish character support */
static void getChar() {
    try {
        int r = in_fp.read();
        if (r != -1) {
            nextChar = (char) r;
            if (Character.isLetter(nextChar) || "çğİöŞÜÇĞİÖŞÜ".indexOf(nextChar) != -1) {
                charClass = HARF;
            } else if (Character.isDigit(nextChar)) {
                charClass = RAKAM;
            } else {
                charClass = BILINMEYEN;
            }
        } else {
            charClass = EOF_CODE;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Whitespace Skipping (getNonBlank):

During analysis, spaces, tabs, and carriage returns are bypassed completely using **isWhitespace(nextChar)** inside a loop, allowing the analyzer to process only meaningful code elements.

```
static void getNonBlank() {  
    while (Character.isWhitespace(nextChar)) {  
        getChar();  
    }  
}
```

Operator Parsing and Lookahead (lookup): When an unknown symbol is encountered, the lookup() function is triggered. For instance, upon detecting an =, the system reads the subsequent character. If the next character is also =, it classifies the token as **ESIT_OP (==)**. If it is a different character, it pushes it back into the input stream using ungetc() and evaluates the initial token as **ATAMA_OP (=)**. This lookahead mechanism is effectively applied to distinguish operators like !=, from !=, <=, and >=.

```
case '<':  
    addChar();  
    next = in_fp.read();  
    if (next == '=') {  
        nextChar = (char) next;  
        addChar();  
        nextToken = KUCUK_ESIT;  
    } else {  
        if (next != -1) in_fp.unread(next);  
        nextToken = KUCUK_OP;  
    }  
    break;  
case '>':  
    addChar();  
    next = in_fp.read();  
    if (next == '=') {  
        nextChar = (char) next;  
        addChar();  
        nextToken = BUYUK_ESIT;  
    } else {  
        if (next != -1) in_fp.unread(next);  
        nextToken = BUYUK_OP;  
    }  
    break;
```

```
if (next == '=') {  
    nextChar = (char) next;  
    addChar();  
    nextToken = DEGIL_ESIT;  
} else {  
    if (next != -1) in_fp.unread(next);  
    nextToken = BILINMEYEN;  
}  
break;
```

Keyword and Identifier Recognition (lex):

If the character class is a letter, subsequent letters and numbers are appended to form a string stored in the lexeme array. This string is compared against reserved keywords (**sayi**, **eger**, **dongu**, **harf**, **degilse**) via strcmp(). If a match occurs, the corresponding Token Code is assigned; otherwise, the general **KIMLIK** code is assigned, indicating a user defined identifier.

```
switch (charClass) {
    case H__ARF:
        addChar();
        getChar();
        while (charClass == H__ARF || charClass == R__AKAM) {
            addChar();
            getChar();
        }
        String k__elime = lexeme.toString();

        // Keywords are updated with full Turkish characters
        if (k__elime.equals("s__ay__i")) nextToken = S__AYI_KW;
        else if (k__elime.equals("e__ğ__er")) nextToken = EGER_KW;
        else if (k__elime.equals("d__ö__ng__ü")) nextToken = D__ONGU_KW;
        else if (k__elime.equals("h__ar__f")) nextToken = H__ARF_KW;
        else if (k__elime.equals("d__e__ğ__il__se")) nextToken = D__EGILSE_KW;
        else if (k__elime.equals("y__az__d__ı__r")) nextToken = YAZDIR;
        else nextToken = KIMLIK;
```

6. SYNTAX ANALYZER DESIGN

The syntax analyzer implements top-down recursive-descent parsing to evaluate the token sequences generated by the lexical analyzer. Each left-hand side rule in the BNF grammar correlates directly with a Java function.

Process Flow: The parser initiates execution by calling the **program()** function. This function checks if the sequence begins with the identifier (**KIMLIK**) "**ana**", followed strictly by **(, , and {**. Afterward, the parser enters the **komutlar()** function to loop through and process internal statements.

```

static void program() {
    System.out.println("Giriş <program>");
    if (nextToken == KIMLIK && lexeme.toString().equals("ana")) {
        lex();
        if (nextToken == SOL_PRTZ) {
            lex();
            if (nextToken == SAG_PRTZ) {
                lex();
                if (nextToken == SOL_SUSLU) {
                    lex();
                    komutlar();
                    if (nextToken == SAG_SUSLU) {
                        lex();
                    } else { hata("Program sonunda ')' eksik!"); }
                } else { hata("Program başlangıcında '{' eksik!"); }
            } else { hata("ana() ifadesinde ')' eksik!"); }
        } else { hata("ana() ifadesinde '(' eksik!"); }
    } else { hata("Program 'ana()' fonksiyonu ile başlamalıdır!"); }
    System.out.println("Çıkış <program>");
}

static void komutlar() {
    System.out.println("Giriş <komutlar>");
    while (nextToken != SAG_SUSLU && nextToken != EOF_CODE) {
        komut();
    }
    System.out.println("Çıkış <komutlar>");
}

```

Expression Precedence: Mathematical evaluations are strictly stratified. Low precedence operations (+, -) are processed in the **ifade()** function, while higher precedence operations (*, /) are managed inside **terim()**. The highest precedence—parentheses, constants, and identifiers—are encapsulated within the **faktör()** function, fully enforcing the mathematical order of operations.

```

static void ifade() {
    System.out.println("Giriş <ifade>");
    terim();
    while (nextToken == TOPLA_OP || nextToken == CIKAR_OP) {
        lex();
        terim();
    }
    System.out.println("Çıkış <ifade>");
}

static void terim() {
    System.out.println("Giriş <terim>");
    faktör();
    while (nextToken == CARP_OP || nextToken == BOL_OP) {
        lex();
        faktör();
    }
    System.out.println("Çıkış <terim>");
}

static void faktör() {
    System.out.println("Giriş <faktör>");
    if (nextToken == KIMLIK || nextToken == TAMSAYI_SBT) {
        lex();
    } else {
        if (nextToken == SOL_PRTZ) {
            lex();
            ifade();
            if (nextToken == SAG_PRTZ) {
                lex();
            } else {
                hata("Sağ parantez ')' eksik!");
            }
        }
    }
}

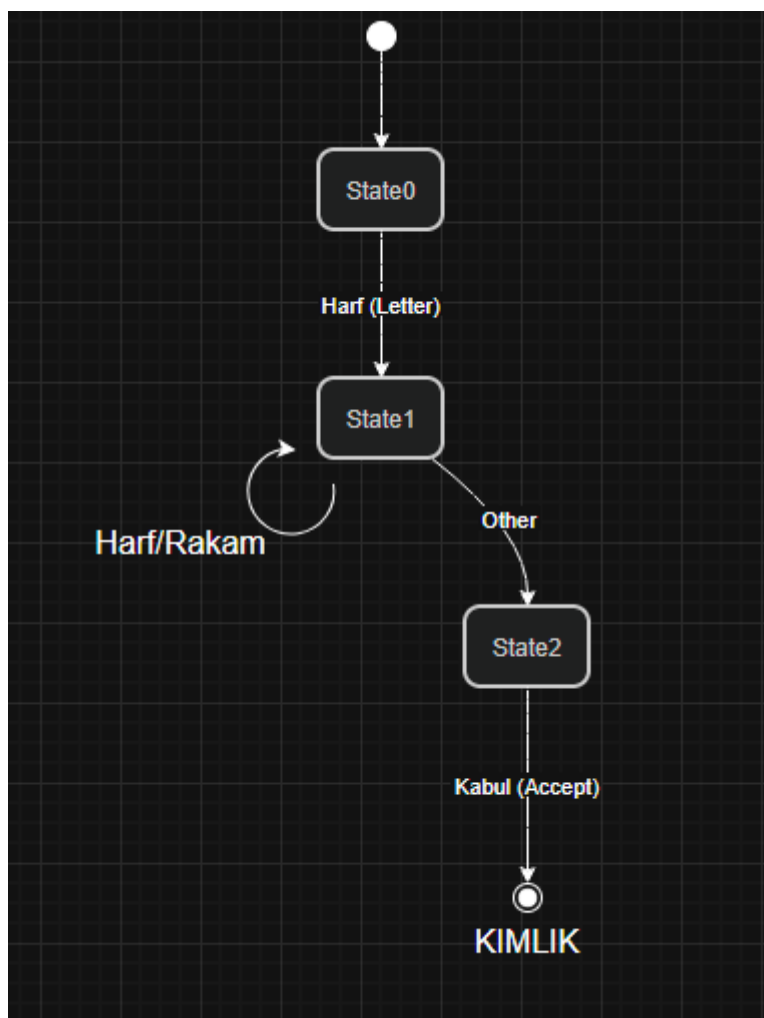
```

7. STATE DIAGRAMS

This section presents the state diagrams and syntax flowcharts for the key lexical structures and control statements of the designed programming language. The diagrams illustrate both the Deterministic Finite Automaton (DFA) models used by the lexical analyzer for token recognition and the parsing flow used by the syntax analyzer.

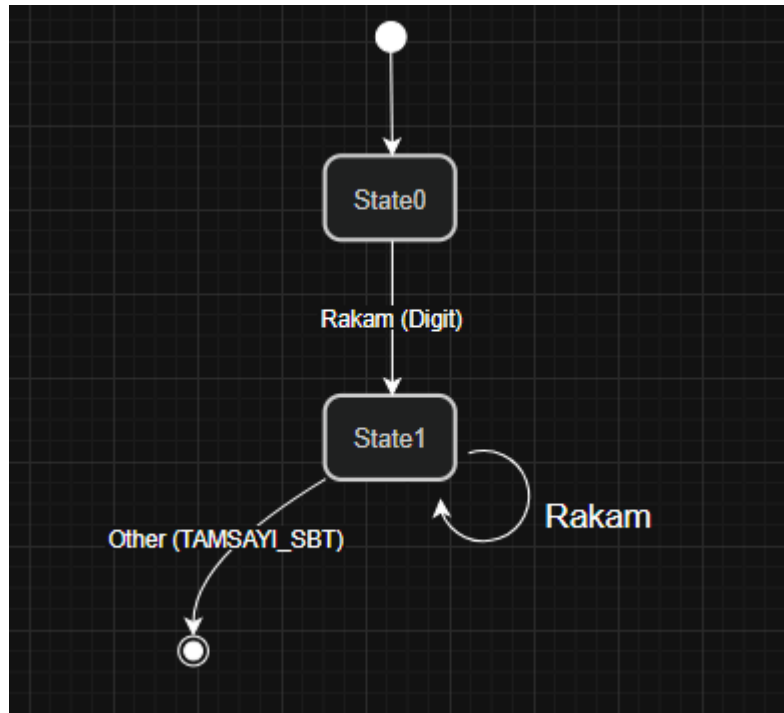
1-Identifier Recognition

This state diagram demonstrates the DFA used to recognize identifiers (**KIMLIK**) in the language. The state machine requires an initial letter to transition to the valid state, followed by a loop that accepts any combination of letters or digits. The machine halts and accepts the token when a different character is encountered.



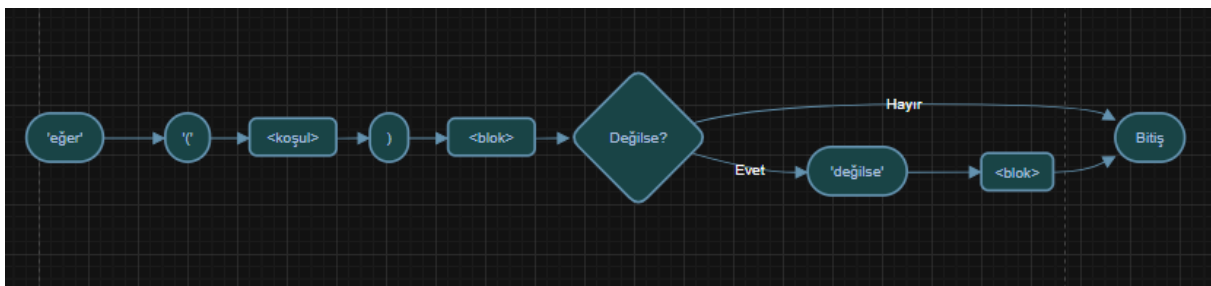
2.Integer Literal Recognition

This DFA outlines the tokenization process for integer literals (**TAMSAYI_SBT**). The machine transitions to an active state upon reading the first digit and loops for all consecutive digits. Any non-digit character terminates the loop, successfully generating the integer token.



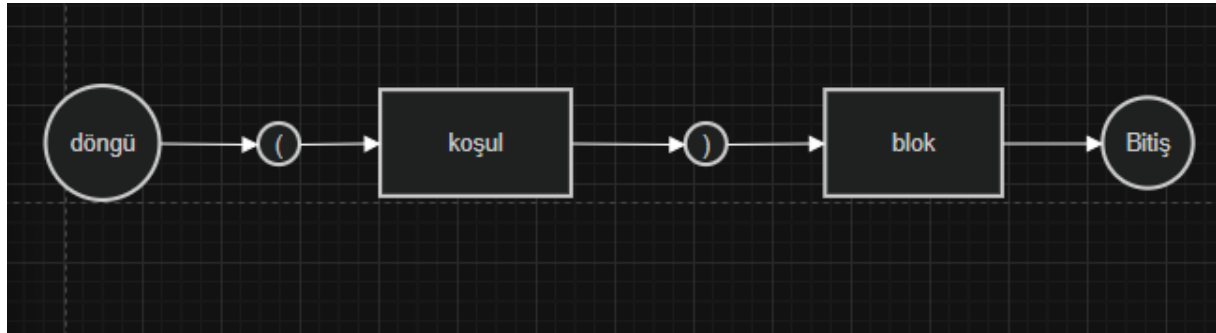
3. Conditional Structure (eğer_stmt)

The following syntax diagram represents the flow of the conditional if-else structure in the parser. It strictly defines the expected sequence of terminal tokens (such as '**eğer**', '(', ')', and optionally '**değilse**') and non-terminal rules (<koşul> and <blok>).



4. Loop Structure (döngü_stmt)

This syntax diagram illustrates the strict parsing order for the loop statement. The parser expects the 'döngü' keyword, followed by the condition enclosed in parentheses, and finally the block of statements to be executed within the loop.



8. SYMBOL TABLE MANAGEMENT

```
/* --- SYMBOL TABLE INFRASTRUCTURE --- */
static class Sembol {
    String isim;
    int tip;
    Sembol(String isim, int tip) {
        this.isim = isim;
        this.tip = tip;
    }
}

static ArrayList<Sembol> sembolTablosu = new ArrayList<>();

static void sembolEkle(String isim, int tip) {
    for (Sembol s : sembolTablosu) {
        if (s.isim.equals(isim)) {
            System.out.println("HATA: '" + isim + "' değişkeni zaten tanımlanmış!");
            return;
        }
    }
    sembolTablosu.add(new Sembol(isim, tip));
    System.out.println("Sembol Tablosuna Kaydedildi -> İsim: " + isim + ", Tip: " + tip);
}
```

Symbol Table Infrastructure:

A dynamic symbol table using Java's ArrayList tracks declarations and prevents duplicates without capacity limits.

- Records are defined by a Sembol class containing the variable's name (**isim**) and type (**tip**).
- New declarations are processed by the **sembolEkle(String isim, int tip)** method.

- The method checks for existing entries using `.equals()`. If a duplicate exists, it throws: **"HATA: '[isim]' değişkeni zaten tanımlanmış!"**.
- If the identifier is unique, a new Sembol object is appended to the list and logged.

Keyword and Identifier Distinction: The system strictly distinguishes between reserved keywords and user-defined identifiers during the lexical analysis phase. Before a token is classified as a general identifier (**KİMLİK**), the generated lexeme string is strictly compared against reserved Turkish keywords (such as **sayı**, **eğer**, **döngü**). Because this categorization happens beforehand, the **Symbol Table** is completely shielded from keywords and only ever processes and stores valid user-defined variables (identifiers) during the **tanımlama()** function.

9.SAMPLE PROGRAMS AND TEST RESULTS

Valid Test Files

These four files successfully compile without syntax errors.

File 1: Variable declarations and assignments

```
ana(){  
  
    sayı x ;  
    sayı y;  
  
    x=13;  
    y=28;  
  
    yazdır(x);  
    yazdır(y);  
  
}
```

Result: Token output correctly parses variable declarations (sayı), identifiers (x, y), assignments, and math operators ending with an EOF exit code 0.

A part of output:

```
Sonraki token: 11, Sonraki kelime: ana  
Giriş <program>  
Sonraki token: 25, Sonraki kelime: (  
Sonraki token: 26, Sonraki kelime: )  
Sonraki token: 50, Sonraki kelime: {  
Sonraki token: 30, Sonraki kelime: sayı  
Giriş <komutlar>  
Giriş <komut>  
Giriş <tanımlama>  
Sonraki token: 11, Sonraki kelime: x  
Sembol Tablosuna Kaydedildi -> İsim: x, Tip: 30  
Sonraki token: 35, Sonraki kelime: ;  
Sonraki token: 30, Sonraki kelime: sayı  
Çıkış <tanımlama>  
Çıkış <komut>  
Giriş <komut>  
Giriş <tanımlama>
```


File 2: Arithmetic expressions

```
ana() {  
    sayı a;  
    sayı b;  
    sayı sonuç;  
  
    a = 10;  
    b = 2;  
    sonuç = a + b * 4 - (a / b);  
    yazdır(sonuç);  
}
```

Result: Parser correctly steps into ifade, terim, and faktor levels, verifying parenthetical precedence and successfully ending with EOF exit code 0.

A part of output:

```
Sonraki token: 11, Sonraki kelime: ana  
Giriş <program>  
Sonraki token: 25, Sonraki kelime: (  
Sonraki token: 26, Sonraki kelime: )  
Sonraki token: 50, Sonraki kelime: {  
Sonraki token: 30, Sonraki kelime: sayı  
Giriş <komutlar>  
Giriş <komut>  
Giriş <tanımlama>  
Sonraki token: 11, Sonraki kelime: x  
Sembol Tablosuna Kaydedildi -> İsim: x, Tip: 30  
Sonraki token: 35, Sonraki kelime: ;  
Sonraki token: 30, Sonraki kelime: sayı  
Çıkış <tanımlama>  
Çıkış <komut>  
Giriş <komut>  
Giriş <tanımlama>  
Sonraki token: 11, Sonraki kelime: y  
Sembol Tablosuna Kaydedildi -> İsim: y, Tip: 30  
Sonraki token: 35, Sonraki kelime: ;  
Sonraki token: 11, Sonraki kelime: x  
Çıkış <tanımlama>  
Çıkış <komut>  
Giriş <komut>  
Giriş <atama>  
Sonraki token: 20, Sonraki kelime: =  
Sonraki token: 10, Sonraki kelime: 15
```

File 3: Conditional statements

```
ana() {  
    sayı x;  
    sayı y;  
  
    x = 45;  
    y = 20;  
  
    eğer (x >= y) {  
        yazdır(x);  
    } değilse{  
        yazdır(y);  
    }  
}
```

Result: Successfully enters , processes the $\geq$ relational operator inside , and maps the alternative under değilse without issue.

A part of output:

```
Sonraki token: 31, Sonraki kelime: eğer  
Çıkış <atama>  
Çıkış <komut>  
Giriş <komut>  
Giriş <eğer_stmt>  
Sonraki token: 25, Sonraki kelime: (  
Sonraki token: 11, Sonraki kelime: x  
Giriş <koşul>  
Giriş <ifade>  
Giriş <terim>  
Giriş <faktör>  
Sonraki token: 43, Sonraki kelime: >=  
Çıkış <faktör>  
Çıkış <terim>  
Çıkış <ifade>
```

File 4: Loop structures

```
ana() {  
    sayı sayaç;  
    sayı sınır;  
  
    sayaç = 0;  
    sınır = 5;  
  
    döngü (sayaç < sınır) {  
        yazdır(sayaç);  
        sayaç = sayaç + 1;  
    }  
}
```

Result: Successfully enters , evaluates the , and processes the iterative reassignment properly.

A part of output:

```
Sonraki token: 11, Sonraki kelime: sınır
Çıkış <atama>
Çıkış <komut>
Giriş <komut>
Giriş <atama>
Sonraki token: 20, Sonraki kelime: =
Sonraki token: 10, Sonraki kelime: 5
Giriş <ifade>
Giriş <terim>
Giriş <faktör>
Sonraki token: 35, Sonraki kelime: ;
Çıkış <faktör>
Çıkış <terim>
Çıkış <ifade>
Sonraki token: 32, Sonraki kelime: döngü
Çıkış <atama>
Çıkış <komut>
Giriş <komut>
Giriş <döngü_stmt>
Sonraki token: 25, Sonraki kelime: (
Sonraki token: 11, Sonraki kelime: sayac
Giriş <koşul>
Giriş <ifade>
Giriş <terim>
Giriş <faktör>
Sonraki token: 44, Sonraki kelime: <
```

Invalid Test File File 5: Intentional Syntax Error

```
4 | ana() {
    sayı a;
    sayı b;

    a = 10;
    b = 20;

    eğer (a == b {
        yazdır(a);
    }
}
```

Result: The compiler halts gracefully. Output: **HATA: Eğer koşulunda ')' eksik!** (Process finished with exit code 1) due to the missing right parenthesis in the conditional if-statement block.

A part of output:

```
Sonraki token: 11, Sonraki kelime: a
Giris <koşul>
Giris <ifade>
Giris <terim>
Giris <faktör>
Sonraki token: 40, Sonraki kelime: ==
Cikis <faktör>
Cikis <terim>
Cikis <ifade>
Sonraki token: 11, Sonraki kelime: b
Giris <ifade>
Giris <terim>
Giris <faktör>
Sonraki token: 50, Sonraki kelime: {
Cikis <faktör>
Cikis <terim>
Cikis <ifade>
Cikis <koşul>
HATA: Eğer koşulunda ')' eksik!
```

10.CONCLUSION

This project successfully delivered the design and implementation of a custom, Turkish-keyword-based programming language, including a fully functional lexical analyzer and a top-down recursive-descent parser written in Java. The architectural implementation was developed by systematically extending the base theoretical framework provided in Chapter 4 of the reference textbook.

The lexical analyzer was engineered to recognize Turkish character sets and deterministically distinguish between reserved keywords and user-defined identifiers. Advanced lookahead logic was integrated into the lexical structure to accurately parse two-character relational operators, such as equality and inequality symbols. On the syntactic level, the recursive-descent parser strictly enforced the language's formal BNF grammar. Mathematical precedence and structural integrity were maintained through a deterministic hierarchy of parsing functions, specifically handling expressions, terms, and factors.

Furthermore, structural consistency was ensured through the implementation of a dynamic Symbol Table, which actively managed variable memory allocation and prevented duplicate identifier declarations during the parsing process. The system's operational robustness was verified through execution against four valid source files, demonstrating accurate processing of variable declarations, arithmetic expressions, conditional branches, and loop structures. Additionally, the compiler's error-handling mechanism successfully intercepted and reported a deliberate syntax violation—a missing right parenthesis in a conditional statement—in the designated invalid test file, gracefully halting the execution.

Ultimately, this implementation fulfilled all specified requirements, successfully bridging formal language definition, token generation, and syntactic validation within a stable compiler front-end architecture.

